PaperDatabase

Une base de connaissance textuelle et vérifiable
pour la lecture assistée d'articles scientifiques

Léonard Rivals Maksym Lytvynenko

Octobre 2026

Le problème

Préparer une thèse = lire, comparer et citer des dizaines d'articles. Un LLM peut aider, mais sur des PDF bruts il travaille mal :

- il relit le texte intégral à chaque question (coût, contexte perdu) ;
- il ne sait pas où chercher un chiffre (tableau ? figure ? annexe ?) ;
- il mélange ce que dit l'article et ce qu'il en déduit ;
- il peut inventer une référence ou une valeur plausible.

Objectif
Des réponses **sourcées et vérifiables**, à faible coût en tokens, sur n'importe quel corpus.

L'idée

Convertir chaque PDF en **fichiers texte structurés** :

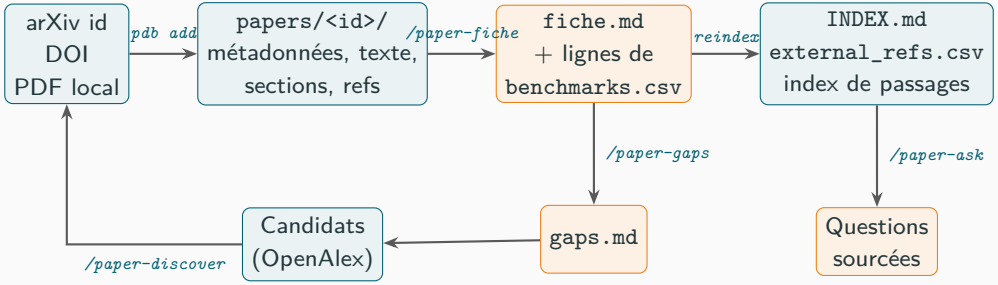
- métadonnées vérifiées, sections numérotées et paginées ;
- références extraites ;
- fiche de lecture et chiffres rangés dans un CSV.

Le LLM (Claude Code) lit la couche la plus haute qui répond, et ne descend que si nécessaire.

Pas de base de données
La source de vérité est une arborescence de fichiers texte : **lisible, greppable, diffable, réparable à la main**.

Par-dessus
Un index de passages local (BM25 + embeddings), **régénérable**, qui ne modifie aucun fichier.

Vue d'ensemble



Qui fait quoi

Qui	Fait quoi
Vous	choisir les articles, déposer les PDF sous paywall, valider les ajouts et les changements de profil, relire une fiche contre le PDF
Les scripts (`bin/pdb`)	télécharger, extraire (GROBID), indexer, vérifier la cohérence, chercher des candidats — sans LLM
Claude (`/paper-...`)	lire, ficher, chiffrer, comparer, relever les gaps, trier les candidats — toujours en citant sa source

Comment ça marche

1. Acquisition et extraction

`bin/pdb add`

- arXiv : API + PDF ;
- DOI : version arXiv trouvée via **OpenAlex** ;
- PDF local, avec export `.bib`/`.ris` éditeur ;
- lots : `--from liste.txt`.

Aucun contournement de paywall : ces PDF se déposent à la main dans `inbox/`.

`ingest` : **GROBID (Docker)**

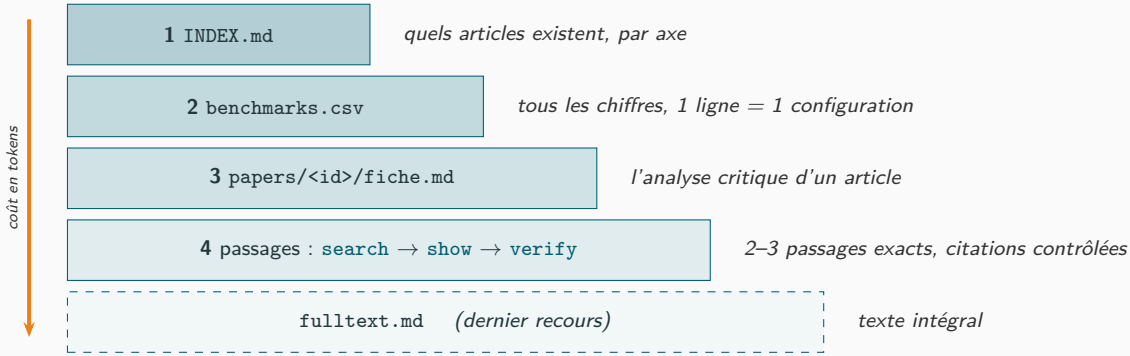
- fusion des métadonnées (`.bib` > arXiv > GROBID ; inconnu = `null`) ;
- nettoyage : en-têtes, n^{os} de page, ligatures perdues ;
- tableaux : PyMuPDF, puis Docling (+ OCR), marqués « à vérifier » ;
- repli PyMuPDF si GROBID est absent ;
- idempotent ; `--reparse` sans GROBID.

2. Un article = un dossier

Fichier	Contenu
`metadata.json`	titre, auteurs, année, venue, DOI/arXiv, preprint signalé, axes, tags, statut
`fulltext.md`	texte intégral nettoyé, sections avec leur numérotation d'origine
`sections.jsonl`	une ligne par section : type (abstract, section, figure, table) et page
`refs.jsonl`	références citées : titre, auteurs, année, DOI, arXiv id
`fiche.md`	fiche de lecture : problème, méthode, résultats, limites, pertinence, lu vs déduit
`claims.jsonl`	preuves atomiques : une affirmation par ligne, liée à un passage vérifié
`paper.pdf`	la preuve d'origine (non versionnée)

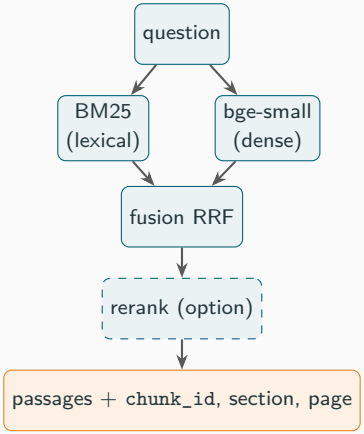
Identifiant stable : `arxiv-2507.18174-real-time-object-detection-classification`

3. Lire du résumé vers le détail



À côté : `gaps.md` (zones non couvertes, chacune appuyée par des `paper_id`), `external_refs.csv` (travaux absents les plus cités).

4. Recherche de passages (style RAG, sans génération)



- `search` : passages classés, rien n'est généré ;
- `show <chunk_id>` : texte exact du passage ;
- `verify <chunk_id> "..."` : la citation y figure-t-elle, nombres compris ?
- `bench`, `paper`, `cites`, `compare` : requêtes sans LLM.

Essai réel : ∼**2 500 tokens** lus au lieu des textes intégraux.

Route GPU optionnelle : Qwen3-Embedding / Reranker-4B, mêmes `chunk_id`.

Exemple : `pdb search`

```
$ bin/pdb search "power measurement method on FPGA board" -k 3
chunk             section                    page  snippet
2015-zhang#039.0  Figures et tableaux          9  Figure 14: «Power» «measurement» of on-«
    board» execution...
2024-zhang#019.2  5.1. Experimental setup     17  ...The «FPGA»'s «power» consumption was
    reported by the
                                                  «power» analyzer of Vivado, as in previous
    work [9]...
2304.11580#005.2  4.1. Experimental Setup      7  ...For the «measurement» of the inference
    time...
```

Puis, dans la réponse de `/paper-ask` :

```
Verdict : partiel
  ✓ méthode de mesure (Vivado power analyzer)          couvert  [2024-zhang#019.2, p. 17]
  ~ puissance carte complète vs. puce                  partiel
  × mesure au wattmètre externe sur plusieurs cartes   absent
```

Sortie de `search` réelle ; réponse abrégée, à titre d'illustration. Chaque citation est passée par `verify` avant d'être écrite : ce qui est déduit est signalé comme tel.

5. Principes de fiabilité

- **Tout est du texte**, sauf le PDF gardé comme preuve.
- **Chaque chiffre est traçable** : section, tableau ou figure source, `chunk_id`.
- `n/r` pour une valeur non rapportée, jamais d'estimation.
- **Lu vs déduit** : section « Statut de lecture » obligatoire dans chaque fiche.
- **Comparaisons honnêtes** : `bench --comparable` (même tâche, jeu, split, métrique).
- **Un seul schéma** : `lib/schema.py`, validé à chaque réindexation.

Règle d'or
Ne jamais inventer de référence. Un preprint est signalé comme tel ; une valeur `confidence: low` aussi.

6. Les skills Claude Code

`/paper-add`	acquisition, ingestion, réindexation
`/paper-fiche`	fiche de lecture + lignes de `benchmarks.csv`, chaque chiffre avec sa source
`/paper-extract`	10 à 40 preuves atomiques (`claims.jsonl`), chacune vérifiée
`/paper-ask`	réponse sourcée : verdict, checklist couvert / partiel / absent
`/paper-compare`	matrice, accords, désaccords, comparaisons impossibles
`/paper-discover`	candidats (autres bases, puis OpenAlex), ajout après accord
`/paper-gaps`	carte des zones non couvertes (`gaps.md`)
`/base-profile`	affine axes et périmètre d'une base — propose un diff, vous validez
`/table-review`	relit les tableaux extraits contre l'image du PDF

7. Faire grandir le corpus

`pdb discover` / `/paper-discover` :

- **en arrière** : ce que le corpus cite ;
- **en avant** : ce qui cite le corpus (`--since 2020`) ;
- **par axe** maigre ou **par gap** ;
- d'abord dans **mes autres bases**.

Sortie : `candidates.csv` avec `in_topic`, raisons (« cité par 3 papiers du corpus »), axe probable.

Mieux vaut 20 fiches solides que 200 PDF non lus

Ajouter un papier seulement s'il comble un axe, étaye un gap ou complète une lignée. **Claude n'ajoute rien sans accord.**

8. Plusieurs bases, un seul code

Une base = un dossier `bases/<nom>/` :

- `base.json` : axes, règles de classement ;
- `profile.md` : thème, périmètre ;
- ses papiers, chiffres, gaps, index.

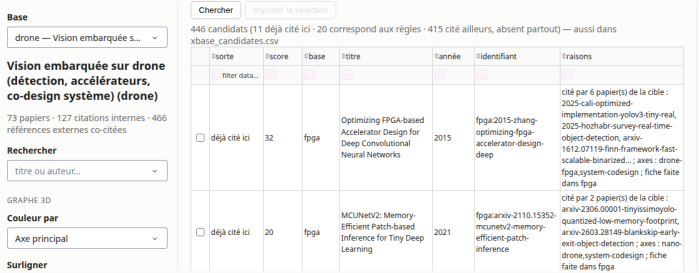
```
pdb base new | scan | import
pdb --base drone bench ...
```

Base	Papiers	Fiches	Benchmarks
`fpga`	104	104	299
`drone`	87	86	352
`stage`	56	56	100
`yolo-data`	44	44	425

Au 18/10/2026. `fpga` : vision embarquée sur FPGA (thèse) ; `drone` : vision embarquée sur drone ; `stage` : apprentissage continu sur MCU ; `yolo-data` : préparation des données YOLO.

Bases indépendantes (index, caches, candidats propres) ; un article se copie de l'une à l'autre sans refaire l'extraction.

Visualisation (pour l'humain, pas pour le LLM)



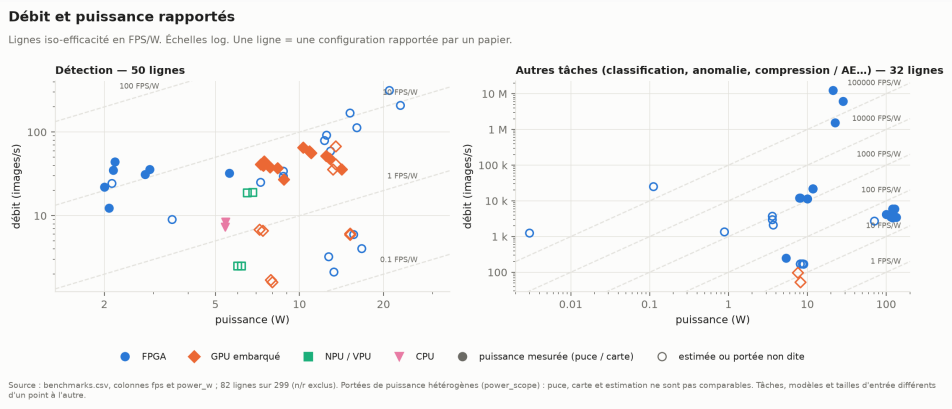
Onglet Bases : articles de la base `fpga` déjà cités par la base `drone`.

`pdb viz` (Dash, local) :

- **Graphe 3D** : citations, similarité, thèmes ;
- **Généalogie** : frise le long des citations ;
- **Bases** : effectifs, profil, import depuis les autres bases.

N'écrit que sur action explicite (créer une base, importer).

Ce que la base produit : exemple `fpga`



`pdb plot` trace directement `benchmarks.csv` (FPS vs puissance, précision vs latence, ressources FPGA, GOPS/W, quantification, couverture par axe). Chaque point renvoie à un papier et à sa section source.

Évaluation et limites

Mesuré par le banc `pdb eval`

Recherche (recall@5)

- **0,90** anglais, jeu de réglage
- **0,84** jeu jamais vu
- **0,50** en français

`verify`

- **38/38** citations au chiffre modifié rejetées
- **40/40** citations exactes acceptées

Extraction

- **91 %** du texte du PDF retrouvé
- **90 %** des tableaux avec leur corps

- Réponses de `/paper-ask` notées à l'aveugle (`/eval-answers`) : hors corpus reconnu 6/6, verdict juste 0,83 (dev) et 0,92 (holdout).
- 288 tests unitaires ; `pdb eval --compare` avant/après tout changement du pipeline.

Limites et suite

- **Tableaux** : les tableaux reconstruits restent à relire contre le PDF ; une valeur de `benchmarks.csv` n'est retrouvée telle quelle que dans 80 % des cas.
- **Absence de sujet** : `search` renvoie toujours des passages ; aucun score ne dit de façon fiable que la base ne traite pas une question.
- **Généricité** : colonnes de `benchmarks.csv` pensées pour le matériel.
- **Coût** : `/paper-fiche` lit encore `fulltext.md` en entier (digest déterministe prévu).
- **Traçabilité automatique** : lier chaque ligne de benchmark à un `chunk_id` vérifié.

Démarrage rapide

Prérequis : Python 3.12, Docker, Claude Code.

```
bash bin/setup.sh                          # .venv + dépendances
bin/pdb grobid start                       # GROBID sur localhost:8070
bin/pdb base new mon-sujet --theme "..."  # une base par sujet
cd bases/mon-sujet && bin/pdb status       # état et prochaines actions
bin/pdb add 2507.18174                     # arXiv id, DOI ou PDF local
```

Puis, dans Claude Code ouvert dans la base :

```
/base-profile                              # axes et périmètre (vous validez)
/paper-fiche <paper_id>                    # fiche + chiffres sourcés
/paper-ask "Quels papiers dépassent 25 FPS sous 5 W ?"
/paper-discover                            # la suite à lire
```

Guide pas à pas : `docs/GUIDE.md` — détails : `SETUP.md`, `DATA_MODEL.md`.

Des PDF $\rightarrow$ du texte structuré $\rightarrow$ des réponses sourcées

Questions ?